Multi-Agent Systems – Course Project Report

Dumbocracy

Campaigning on a Social Network

Cascades, consensus, planning under uncertainty and knowledge, studied with a GPU-batched election game

Submitted by

Name	Roll number
[TODO: Full name] (Kaushik)	AI23BTECH11017
[TODO: Full name] (Madhava Asrith)	[TODO: roll no.]
[TODO: Full name] (Hemanth Reddy)	[TODO: roll no.]
[TODO: Full name] (Vishank Singh)	[TODO: roll no.]
[TODO: Full name]	AI23BTECH11033

Under the guidance of [TODO: instructor name]

Indian Institute of Technology Hyderabad

November 2026

Code repository: https://github.com/saikaushhikp/CLOUT

Abstract

An election campaign is a contest fought on a network. A party spends money to change what people think, but people also change each other's minds, and the party never sees the electorate directly. We built *Dumbocracy*, a single-player election game, to study this situation with the tools of multi-agent systems.

The electorate has between ten thousand and one million simulated voters. They live in four regions and are linked by a friendship network. Each voter wants the government budget split across five sectors in a particular way. The player writes a manifesto (their own budget split) and then has a few campaign rounds to win a majority. Each round they may hire party workers, who start word-of-mouth cascades through the network; buy regional media airings; and pay for survey questions. Meanwhile, every voter drifts towards the average opinion of their friends.

The game connects several course topics in one place. Opinion drift is DeGroot consensus with stubborn agents; party workers spread influence by an Independent Cascade; choosing a manifesto is an optimisation problem on the probability simplex, which we solve with projected gradient and ADMM; and the player's uncertainty about the electorate is described exactly by epistemic logic, with temporal logic describing how a campaign unfolds. We implemented the whole model as one PyTorch code path that runs a single world on a CPU or hundreds of worlds at once on a GPU, together with Bayesian and scripted computer players, a formal-reasoning layer that tells the player after the game what they knew and when, a web interface for human play, and an experiment harness with paired seeds and machine-checked reporting.

This report describes the model, the players, the reasoning layer and the software. The experiment suite built on top of it is reported in Chapter 7.

Contents

Abstract i

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 The game in one paragraph . . . 1
- 1.4 Questions we asked . . . 2
- 1.5 Contributions . . . 2
- 1.6 How this report is organised . . . 2
- 1.7 Use of AI tools . . . 3
- 1.8 Who did what . . . 3
- 1.9 Summary . . . 3

2 Background and Related Work 4
- 2.1 Overview . . . 4
- 2.2 Networks and random graphs . . . 4
- 2.3 Opinion dynamics and consensus . . . 4
- 2.4 Contagion and influence maximisation . . . 5
- 2.5 Voting on a policy space . . . 5
- 2.6 Optimisation on the simplex . . . 5
- 2.7 Planning under uncertainty . . . 6
- 2.8 Epistemic and temporal logic . . . 6
- 2.9 Multi-agent systems and games as instruments . . . 6
- 2.10 Summary . . . 7

3 The Game Model 8
- 3.1 Overview . . . 8
- 3.2 The electorate . . . 8
- 3.3 Voting . . . 9
- 3.4 What the player can do . . . 10
- 3.5 How opinions change . . . 11
- 3.6 What the player can see . . . 11
- 3.7 How the rules evolved . . . 12
- 3.8 Summary . . . 12

4 Players and Decision Making 13
- 4.1 Overview . . . 13
- 4.2 The player interface . . . 13
- 4.3 The full-information reference: ORACLE . . . 13
- 4.4 The Bayesian planner . . . 14
- 4.5 Scripted players . . . 16
- 4.6 An estimate of good play: plan search . . . 16
- 4.7 Summary . . . 16

5 Reasoning About Knowledge and Time **17**
5.1 Overview . . . 17
5.2 Possible worlds . . . 17
5.3 Epistemic properties . . . 17
5.4 Temporal properties . . . 18
5.5 The epistemic debrief . . . 18
5.6 Summary . . . 18

6 System Design and Implementation **20**
6.1 Overview . . . 20
6.2 Architecture . . . 20
6.3 The game engine . . . 20
6.4 Reproducibility . . . 20
6.5 GPU-first simulation . . . 21
6.6 Keeping hidden information hidden . . . 21
6.7 The web game . . . 22
6.8 Testing . . . 22
6.9 Experiment harness . . . 22
6.10 Summary . . . 23

7 Experiments and Results **24**
7.1 Overview . . . 24
7.2 Summary . . . 24

8 Limitations and Future Work **25**
8.1 Overview . . . 25
8.2 Limitations of the model . . . 25
8.3 Limitations of the method . . . 25
8.4 Future work . . . 25
8.5 Summary . . . 26

9 Conclusion **27**
9.1 Overview . . . 27
9.2 What we did . . . 27
9.3 What we learned from building it . . . 27
9.4 Summary . . . 27

References **29**

A Notation and Default Parameters **32**

B Reproducing the Work **34**

List of Figures

3.1 The flow of a game (top) and the order of events inside one campaign round (bottom). 10

4.1 One decision of the Bayesian planner. While a question is worth its price it asks one; otherwise it commits a manifesto (in the survey phase) or chooses a round allocation (in a campaign round). 14

6.1 Software architecture. Players and the reasoning layer see the game only through observations; the experiment harness drives the simulation core directly for speed. 21

List of Tables

1.1 Contribution of each group member. 3

3.1 The public archetype library. Each row is the mean budget split μ of one archetype. 9
3.2 The public personality library. Each voter draws its susceptibilities from the Beta distributions of its region's personality. 9

A.1 Notation used in the report. 32
A.2 Default parameters of the v3 game. 33

Chapter 1

Introduction

1.1 Overview

This chapter explains what problem we set out to study and why a game is a good way to study it. It states the questions that drive the project, lists what we built, and describes how the rest of the report is organised. It ends with a statement of who in the group did what.

1.2 Motivation

Political campaigns spend money to move opinions. Those opinions are not static: people talk to their friends, read the news and slowly converge on what their social circle thinks. A campaign therefore pushes on a system that is already moving by itself. Two classical families of models describe the two halves of this picture.

- **Opinion dynamics** models, such as DeGroot averaging [9], describe how opinions drift towards a consensus when each person repeatedly averages the views of their neighbours. Stubborn agents, people whose view never changes, can stop that consensus from forming [19, 33].
- **Contagion** models, such as the Independent Cascade [20, 27], describe how a new idea jumps from person to person. They have a sharp threshold: below it a cascade dies out, above it a cascade can reach a large part of the network [6].

A real campaign also works in the dark. The party does not know exactly which regions want what, or how easily each region can be persuaded. Every decision is made under uncertainty, and two questions keep coming up: *what do I actually know?* and *can I still guarantee a win?* Epistemic logic gives these questions precise meanings [16], and temporal logic lets us ask when, during a campaign, the outcome is settled [40].

We wanted one setting in which all of these ideas interact, rather than studying each in isolation. A game gives exactly that. It has clear rules, a clear goal, and a player who must act with incomplete information. Because we wrote the simulator ourselves, we can also open the box afterwards: we can replay a game, look at the hidden electorate, and check what the player could and could not have known.

1.3 The game in one paragraph

Dumbocracy is a single-player election game. A synthetic electorate of N voters (between 10^3 and 10^6; 10^4 by default) lives in four regions joined by a friendship network. Every voter has a preferred split of a government budget over five sectors: education, welfare, agriculture, industry and the environment. The player begins with a treasury of 500 coins and some free public information. They may pay for survey questions, then they commit to a manifesto: their own five-sector budget split. After that they play five campaign rounds. In each round they may hire party workers in any region, who try to convert their friends, and buy media airings in any region. Between rounds, voters drift towards their friends' views. At the end there is an

election, and the player wins with more than half the votes. The regions' true preferences and personalities stay hidden throughout.

1.4 Questions we asked

The project is organised around four groups of questions.

Q1. Mechanics against theory. Does the simulated electorate behave as course theory predicts? For example, does consensus speed follow the spectrum of the network, and does the worker cascade show the epidemic threshold that the theory of contagion predicts?

Q2. Good play. Which strategies win? Does a player that gathers information and plans ahead beat simple rules of thumb, and how close do they come to the best achievable play?

Q3. Information and knowledge. How much is a survey question worth? Can the player learn the hidden personalities of regions? When the game tells the player that a win was "guaranteed", how far can that statement be trusted?

Q4. Game design. Is the game balanced, so that no fixed pattern of spending always wins? How much of the outcome is luck, and which design parameters matter most?

1.5 Contributions

The project produced the following.

1. **A formal model** of a campaign on a networked electorate, combining DeGroot consensus, stubborn agents, an Independent Cascade and a probabilistic voting rule, with a precise information model that separates what is public, what can be bought and what stays hidden (Chapter 3).
2. **Computer players** that act only on what a human could see: scripted baselines, a full-information reference called the ORACLE, and a Bayesian planner that chooses survey questions by information gain, optimises the manifesto on the simplex, and plans spending by simulated rollouts (Chapter 4).
3. **A formal-reasoning layer** that evaluates epistemic statements ("a win was possible", "a win was guaranteed") over sampled worlds consistent with the player's knowledge, and temporal statements over the course of a game. It drives a post-game debrief for human players (Chapter 5).
4. **A GPU-first implementation**: one PyTorch code path that is bit-identical on the CPU to the reference simulator and runs many worlds in parallel on a GPU, together with a web interface and an experiment harness with paired seeds and automatic checking of every reported number (Chapter 6).
5. **A pre-registered experiment suite** of 23 experiments that test the game against theory and against its own design goals (Chapter 7).

1.6 How this report is organised

Chapter 2 gives the background needed to read the rest of the report: opinion dynamics, contagion and influence maximisation, optimisation on the simplex, planning under uncertainty, and epistemic and temporal logic. Chapter 3 defines the game precisely. Chapter 4 describes the computer players, and Chapter 5 the formal-reasoning layer. Chapter 6 describes the software:

the engine, GPU batching, reproducibility, the web game and the experiment harness. Chapter 7 reports the experiments. Chapter 8 discusses limitations and future work, and Chapter 9 concludes. The appendices list the notation, the default parameters and the commands needed to reproduce the work. All code is public at https://github.com/saikaushhikp/CLOUT.

1.7 Use of AI tools

We used AI coding assistants while writing parts of the simulator, the tests, the experiment scripts and the plotting code. Every design decision, every rule of the game and every experiment was decided by the group, and all generated code was reviewed and tested before use. The text of this report was written by the group. The group takes joint responsibility for all of its content.

1.8 Who did what

The table below records each member's contribution. **[TODO:** Each member should write their own row in their own words, as the instructions require. The areas listed are the main parts of the project, as a guide.]

Table 1.1: Contribution of each group member.

Member	Contribution
Kaushik	**[TODO:** e.g. game specification, engine, GPU batching, experiment suite, report]
Madhava Asrith	**[TODO:** describe own contribution]
Hemanth Reddy	**[TODO:** describe own contribution]
Vishank Singh	**[TODO:** describe own contribution]
[TODO: Fifth member]	**[TODO:** describe own contribution]

The main parts of the project, for reference, were: the game specification and calibration; electorate generation and the network model; social dynamics, workers, media and voting; the information model and game engine; the computer players and manifesto optimisers; the epistemic and temporal logic layer; the HTTP API and web interface; testing; the experiment suite and analysis; and the writing of this report.

1.9 Summary

Campaigns act on opinions that also move on their own, and they do so without seeing the electorate. We built a game that captures this and used it as a research instrument. The project asks whether the simulated electorate follows course theory, which strategies win, what information is worth, and whether the game is well designed. The next chapter introduces the theory that the rest of the report relies on.

Chapter 2

Background and Related Work

2.1 Overview

The game draws on several areas that are usually taught separately. This chapter introduces each one briefly, with only the results we use later. Readers who know a topic can skip its section. We start with how opinions spread (consensus and contagion), move on to the optimisation methods used to pick a manifesto, and end with planning under uncertainty and the logics of knowledge and time.

2.2 Networks and random graphs

A social network is a graph $G = (V, E)$ whose nodes are people and whose edges are friendships. We write A for its adjacency matrix ($A_{ij} = 1$ when i and j are friends), d_i for the degree of node i, and D for the diagonal matrix of degrees. The *Laplacian* is $L = D - A$. Its second-smallest eigenvalue $\lambda_2(L)$, the algebraic connectivity, measures how well connected the graph is [17].

The simplest random graph is the Erdős–Rényi graph, in which every pair of nodes is joined independently with the same probability [15]. Real societies are not like that: people have more friends in their own community. The *stochastic block model* captures this by splitting nodes into blocks and letting the link probability depend on the pair of blocks [23]. Our electorate uses a stochastic block model whose blocks are the four regions.

2.3 Opinion dynamics and consensus

In the DeGroot model [9], each person holds an opinion x_i and repeatedly replaces it with a weighted average of their neighbours' opinions,

$$x(t+1) = W\,x(t), \qquad W = D^{-1}A, \tag{2.1}$$

where W is row-stochastic. On a connected, non-bipartite graph the opinions converge to a common value. How fast they converge is governed by the second-largest eigenvalue modulus μ_2 of W: the disagreement shrinks roughly like μ_2^t [21, 36]. A graph with weak links between communities has μ_2 close to 1 and converges slowly.

Two extensions matter for us. In the Friedkin–Johnsen model [19], each person keeps some attachment to their initial opinion, so the population settles at an equilibrium instead of a consensus. *Stubborn agents* or *zealots*, who never change their opinion, have a similar effect: a single zealot can pull a whole population towards their view, and zealots with opposing views prevent consensus altogether [1, 33]. Bounded-confidence models [22] are another well-known variant, in which people only listen to opinions close to their own; we do not use them, but they motivate the robustness checks below.

Finally, *resilient consensus* asks how normal agents can still agree when some agents misbehave. The W-MSR algorithm [28] has each agent discard the F most extreme neighbour values before averaging. It succeeds if the graph is sufficiently *robust*, a combinatorial property that is hard to check exactly. In our game, party workers who never change their view are, from

the network's point of view, exactly such misbehaving agents, which makes W-MSR a natural defence to test.

2.4 Contagion and influence maximisation

The *Independent Cascade* model [20, 27] describes how an idea spreads. When a node becomes active, it gets one chance to activate each inactive neighbour j, and succeeds with some probability q_j. Newly activated nodes then get their own chance in the next step.

Whether a cascade fizzles out or spreads widely depends on an *epidemic threshold*. For contagion on a network, the relevant quantity is the spectral radius of the adjacency matrix weighted by the transmission probabilities [6, 38]. If each neighbour j is converted with probability q_j, the expected number of new activations in the next step grows like the operator $A\,D_q$, where $D_q = \operatorname{diag}(q)$. The cascade grows when $\lambda_{\max}(AD_q) > 1$ and dies out otherwise.

Influence maximisation asks which k seed nodes to activate so that the expected spread is largest. Kempe, Kleinberg and Tardos [27] showed that the expected spread under the Independent Cascade is a *submodular* function of the seed set: each new seed adds less than the previous one. For submodular functions, the greedy algorithm is guaranteed to reach at least $1 - 1/e \approx 63\%$ of the optimum [35]. The CELF algorithm [29] makes greedy selection much faster by updating marginal gains lazily. Our party workers are seeds of exactly such a cascade, so these results tell us what to expect from hiring them.

2.5 Voting on a policy space

In the spatial theory of voting [11, 14], each voter has an ideal point in a space of policies, and votes for the candidate whose platform is closest. Our voters' ideal points are budget splits, so the policy space is the probability simplex

$$\Delta = \left\{ x \in \mathbb{R}^5 : x_k \geq 0,\ \textstyle\sum_k x_k = 1 \right\}. \tag{2.2}$$

We measure the distance between two budget splits by the *total-variation* distance $\mathrm{TV}(x, y) = \frac{1}{2}\|x - y\|_1$, which is the share of the budget that would have to move to turn one split into the other. Instead of a hard "closest platform wins" rule, we use a probabilistic vote: a voter supports the party with a probability that rises smoothly as the manifesto gets closer to their ideal point.

2.6 Optimisation on the simplex

Choosing a manifesto means maximising a smooth but non-concave function over Δ. We use four first-order methods.

- **Projected gradient.** Take a gradient step, then project back onto Δ. The Euclidean projection onto the simplex can be computed exactly in $O(n \log n)$ time [12]. It is the *proximal operator* of the simplex's indicator function, so projected gradient is a special case of the proximal gradient method [37].
- **Mirror descent.** Replace the Euclidean geometry with the entropy, which gives multiplicative updates that stay inside the simplex automatically [3].
- **Frank–Wolfe.** Move towards the vertex of the simplex that best aligns with the gradient. No projection is needed [18, 25].

- **ADMM.** Split a problem into pieces that are solved separately and then forced to agree [5]. In *consensus* form, each region keeps its own copy of the manifesto, and a shared variable pulls the copies together. This is the textbook model of distributed optimisation in a multi-agent system.

For black-box search over whole campaign plans we also use the *cross-entropy method*, which repeatedly samples candidate plans, keeps the best fraction and refits the sampling distribution to them [42].

2.7 Planning under uncertainty

A player who does not know the hidden state of the world faces a partially observable decision problem [26]. Solving such problems exactly is intractable at our scale, so we use two standard approximations. First, the player keeps a *belief*: a probability distribution over the possible hidden worlds, updated by Bayes' rule as observations arrive. Second, the player plans by *rollouts* [4]: for each candidate action, it simulates the rest of the game in several worlds sampled from its belief, follows a fixed default policy afterwards, and picks the action with the best average result.

Information has a price in our game, so the player must also decide which question to ask. Bayesian experimental design suggests asking the question whose answer is expected to reduce uncertainty the most, that is, the question with the largest expected information gain (mutual information between the hidden state and the answer) [7, 30].

2.8 Epistemic and temporal logic

Epistemic logic [16] formalises knowledge. An agent considers a set of worlds possible given what it has observed. It *knows* a statement φ, written $\mathsf{K}\varphi$, if φ is true in every world it considers possible, and it considers φ *possible*, written $\mathsf{M}\varphi$, if φ is true in at least one of them; the two are dual, $\mathsf{M}\varphi \equiv \neg\mathsf{K}\neg\varphi$. When the "considers possible" relation is an equivalence relation, the logic is called S5. This fits a player who remembers everything they observed: two worlds are indistinguishable exactly when they would have produced the same observations. A subtle but important distinction is between *knowing that* a winning plan exists (in each possible world some plan wins, but perhaps a different one) and *knowing how* to win (one plan wins in every possible world).

Temporal logic describes how properties change over time. Linear temporal logic (LTL) [40] has operators such as "eventually" ($\Diamond$), "always" ($\Box$) and "until". Computation tree logic, or CTL [8], quantifies over branching futures ("on some path" and "on all paths"), and alternating-time temporal logic [2] asks whether an agent has a strategy that forces a property. Signal temporal logic (STL) [31] adds real-valued signals, and its *robustness* semantics [10] measures by how much a property is satisfied or violated, not only whether it is. In our game, the vote margin is such a signal.

2.9 Multi-agent systems and games as instruments

The game is a multi-agent system in the sense of [43, 47]: thousands of simple agents (voters) follow local rules, and one strategic agent (the party) tries to steer the collective outcome while reasoning about what it knows. Studying such systems through simulation is common, and

textbooks such as [13] use small network models to explain how behaviour spreads. What is less common is to put the full set of tools, from spectral graph theory to epistemic logic, around one model, and to check each prediction with controlled experiments. That is the aim of this project.

2.10 Summary

Opinions in our game drift by DeGroot consensus, slowed by stubborn agents and by weak links between regions. Party workers spread influence by an Independent Cascade, which has a spectral threshold and diminishing returns. The manifesto is chosen by optimisation on the simplex, using projected gradient, mirror descent, Frank–Wolfe or ADMM. The player plans with a Bayesian belief and rollouts, asks questions by information gain, and its knowledge is described by S5 epistemic logic and its progress by temporal logic. The next chapter defines the game itself.

Chapter 3

The Game Model

3.1 Overview

This chapter defines the game precisely. We describe how an electorate is generated, how voters change their minds, what the player can do, how the election is decided and what the player is allowed to see. Every rule in this chapter is implemented exactly as stated in the code, and the default values are those used in the third version (v3) of the game. Table A.2 in Appendix A collects all default parameters in one place.

3.2 The electorate

3.2.1 Regions

The N voters are split into four regions, called North, South, East and West. Every region holds at least 15% of the voters, and the rest are split at random. Concretely, with $m = \lceil 0.15N \rceil$,

$$\pi \sim \mathrm{Dirichlet}(2,2,2,2), \qquad n_r = m + \left[\mathrm{Multinomial}(N - 4m,\ \pi)\right]_r, \tag{3.1}$$

so that $\sum_r n_r = N$ and $n_r \geq m$ for every region r.

3.2.2 The friendship network

Friendships form a stochastic block model whose blocks are the regions. For each game we draw ten connection probabilities: one inside each region, $p_r \sim U[0.5, 0.9]$, and one for each pair of regions, $p_{rs} \sim U[0.01, 0.2]$. Using them directly would make the graph almost complete, so we scale all of them by a single sparsity factor s that keeps their ratios but sets the expected mean degree to $\bar{k} = 30$:

$$P_{rr} = s\, p_r, \qquad P_{rs} = s\, p_{rs}, \qquad s = \frac{\bar{k}\, N}{\sum_r p_r\, n_r(n_r - 1) + \sum_{r \neq s} p_{rs}\, n_r n_s}. \tag{3.2}$$

Inside each block, every pair of voters is linked independently with probability P. With typical draws about three quarters of a voter's friends live in their own region. Edges are unweighted. The player is told the number of voters in each region and the number of friendships inside and between regions, but never the edges themselves.

3.2.3 What voters want: archetypes

Each voter i holds a preferred budget split $v_i \in \Delta$ over the five sectors, in the order education, welfare, agriculture, industry, environment. Regions differ in what they want. There is a public library of six *archetypes* (Table 3.1), and a hidden one-to-one map σ assigns one archetype to each region; there are $6 \cdot 5 \cdot 4 \cdot 3 = 360$ such maps. A voter in region r starts at

$$v_i(0) \sim \mathrm{Dirichlet}\left(\kappa\, \mu_{\sigma(r)}\right), \qquad \kappa = 60, \tag{3.3}$$

so voters in the same region want similar but not identical things.

Table 3.1: The public archetype library. Each row is the mean budget split μ of one archetype.

ID	Name	Education	Welfare	Agriculture	Industry	Environment
A0	Agrarian heartland	0.15	0.20	0.40	0.10	0.15
A1	Industrial belt	0.15	0.15	0.10	0.45	0.15
A2	Green urban	0.25	0.10	0.10	0.15	0.40
A3	Welfare-reliant	0.15	0.45	0.15	0.10	0.15
A4	Knowledge hub	0.40	0.10	0.10	0.25	0.15
A5	Balanced middle	0.20	0.20	0.20	0.20	0.20

Table 3.2: The public personality library. Each voter draws its susceptibilities from the Beta distributions of its region's personality.

ID	Name	Friends α	Media β	Canvassing γ
P0	Socially driven	Beta(6,3)	Beta(2,5)	Beta(3,4)
P1	Media-savvy	Beta(3,4)	Beta(6,3)	Beta(2,5)
P2	Independent	Beta(2,6)	Beta(2,6)	Beta(2,6)
P3	Receptive to canvassing	Beta(3,4)	Beta(3,4)	Beta(6,3)

3.2.4 How voters can be moved: personalities

Voters also differ in how easily they are moved. Each voter has three susceptibilities in $[0, 1]$: to friends (α_i), to media (β_i) and to canvassing by party workers (γ_i). Each region has one of four public *personalities* (Table 3.2), assigned by a hidden permutation τ (24 possibilities), and voters draw their susceptibilities from that personality's Beta distributions.

Finally, each voter is independently a *zealot* with probability $\zeta = 0.05$. Zealots never change their mind ($\alpha = \beta = \gamma = 0$), but their friends still listen to them.

The pair $\theta = (\sigma, \tau)$ is the *hidden type* of the world; there are $360 \times 24 = 8640$ of them. Everything else (the graph, the individual preferences, the susceptibilities and the zealots) is the *micro-state*. The player's main task in gathering information is to narrow down θ.

3.3 Voting

Voter i's agreement with a manifesto b is one minus the total-variation distance,

$$S_i = 1 - \tfrac{1}{2} \|v_i - b\|_1 \in [0, 1]. \tag{3.4}$$

On election day the voter supports the party with probability

$$P_i = \frac{1}{1 + \exp\big(-\beta_v (S_i - z)\big)}, \qquad \beta_v = 20, \quad z = 0.825. \tag{3.5}$$

A voter whose agreement equals the threshold z votes for the party half the time; the slope β_v controls how sharply support rises around it. We call

$$E = \frac{1}{N} \sum_{i=1}^{N} P_i \tag{3.6}$$

the *expected vote share*. It is the main quantity the player tries to raise, and the one we analyse. The official result is a seeded random draw in which each voter votes independently with probability P_i. The party wins if it receives more than $N/2$ votes; a tie is a loss.

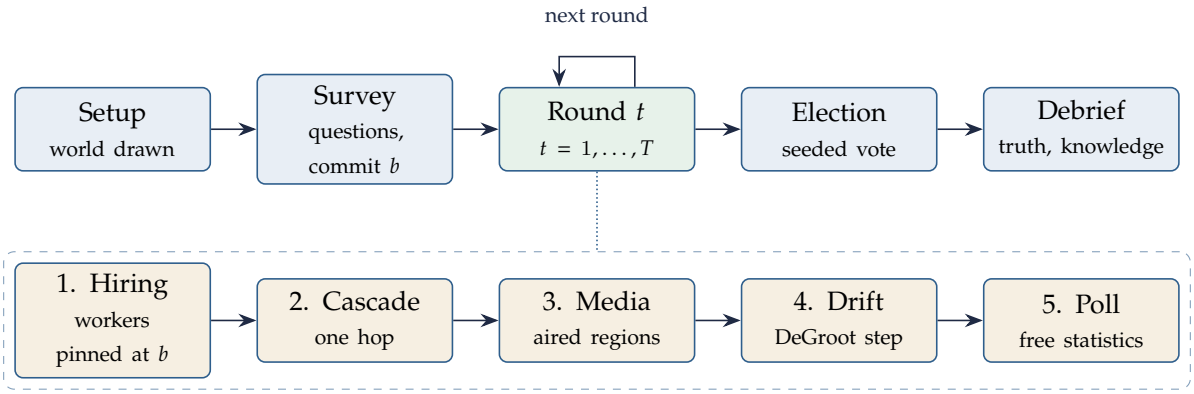


Figure 3.1: The flow of a game (top) and the order of events inside one campaign round (bottom).

3.4 What the player can do

3.4.1 Turn structure

A game has three phases (Fig. 3.1). In the *survey* phase the player may buy survey questions and then commits a manifesto $b \in \Delta$, which is fixed for the rest of the game. In each of the T *campaign rounds* ($T = 5$ by default; the web game allows 3 to 5) the player chooses an allocation: how many workers to hire in each region and which regions to air media in. They may also ask further survey questions. After the last round the *election* is held. The treasury starts at 500 coins (the player may choose up to 1000 coins), and unspent money is worth nothing.

3.4.2 Party workers and the cascade

A worker costs 20 coins, paid once. When the player hires a worker in region r, the game picks a random voter of that region who has at least three friends and is not a zealot or already a worker. That voter's preference is set to the manifesto, $v_i = b$, and stays there for the rest of the game: friends and media never move a worker. In the language of Chapter 2, workers are stubborn agents placed by the party.

Workers also start an Independent Cascade that advances one hop per round:

- The *spreaders* of round t are the workers hired in round t together with the voters converted in round $t - 1$.
- Each spreader tries once to convert each of its friends j (workers and zealots cannot be targeted). If j receives e_j attempts this round, it converts with probability

$$1 - \left(1 - \gamma_j P_j\right)^{e_j}, \tag{3.7}$$

  where P_j is j's current probability of voting for the party, from Eq. (3.5).
- A converted voter jumps to $v_j = b$ and spreads in the next round only. After that it is an ordinary voter again and drifts with its friends.
- No voter can receive more than three attempts in the whole game, failures included.

The factor P_j in Eq. (3.7) is a deliberate design choice: voters who already lean towards the manifesto are easier to convert. It also couples the two halves of the game. A manifesto close to what a region wants raises P_j there, which raises the conversion chance and can push that region's cascade above its epidemic threshold.

3.4.3 Media

Media airings target whole regions. Airing in region r costs $m_0\, n_r/(N/4)$ with $m_0 = 50$, so a quarter-sized region costs 50 coins. Each voter in an aired region is reached with probability $p_m = 0.6$, and a reached voter feels a pull towards b of strength

$$e_i = \eta_m(n)\,\beta_i, \qquad \eta_m(n) = \frac{\eta_0}{1+\lambda n}, \quad \eta_0 = 0.25,\ \lambda = 0.5, \tag{3.8}$$

where n counts the player's earlier airings in that region. Repeated airings in the same region therefore have diminishing returns.

3.5 How opinions change

Once the round's hiring, cascade and media are resolved, every voter updates at the same time. Let $a_i = \omega\,\alpha_i$ with $\omega = 0.15$ be the weight voter i gives to friends, and c_i the combined campaign pull on i this round. The update is

$$v_i \leftarrow (1 - a_i - c_i)\,v_i \;+\; a_i\,\bar{v}_{\mathcal{N}(i)} \;+\; c_i\,b, \qquad \bar{v}_{\mathcal{N}(i)} = \frac{1}{d_i}\sum_{j\in\mathcal{N}(i)} v_j, \tag{3.9}$$

where $\mathcal{N}(i)$ is the set of i's friends. If $a_i + c_i$ would exceed 1, the campaign pull is scaled down so that it does not. The new preference is a convex combination of points of the simplex, so it stays in the simplex. Zealots and workers do not update. The friend term is exactly the DeGroot rule, and with W-MSR switched on (an option we test) the most extreme friends are dropped before averaging.

3.6 What the player can see

The information model separates three kinds of facts.

Public at all times.
The rules and prices, N and T, the size of each region, the archetype and personality libraries (but not σ or τ), the player's own treasury and actions, and a network census (friendship counts inside and between regions).

Free each round.
For every sector, the share of all voters who rank it among their top three, and a national poll of 400 random voters with its margin of error.

Paid, 10 coins each.
A survey question "in region r, what share of voters prefer sector s_2 over sector s_1?" The game answers from a fresh random sample of 10% of the region, with a 95% margin of error. A question may be repeated, and repeated answers within a round are pooled.

The maps σ and τ, the graph, every individual preference and susceptibility, the zealots and all seeds stay hidden. The implementation enforces this with allow-listed serialisers and tests that search every response for hidden fields (Chapter 6).

Everything the player observes is appended to a *knowledge log*. A world is consistent with the log if replaying the player's own actions in that world would have produced the same observations. This set of consistent worlds is the player's *information set*, and it is the foundation of the reasoning layer in Chapter 5.

3.7 How the rules evolved

The rules above are the third version of the game. The first two versions taught us what did not work, and the main changes are worth recording.

- **Calibration.** Early defaults made the game either trivial or hopeless. A calibration study on 200 seeds tuned the voter concentration κ, the social step ω, the media strength, and the voting slope and threshold until a uniform manifesto with no campaign rarely won and a well-informed campaign usually did.
- **Party workers.** In v2, workers were free each round and made random doorstep contacts, followed by a three-hop cascade within the round. In v3 there are no free workers: each worker is bought once, is a real voter of the network, stays pinned at the manifesto, and spreads influence one hop per round. This made workers a long-term investment and tied them to the network's structure.
- **Prices and choices.** The worker price was set to 20 coins and the survey price to 10 coins; the player may choose a treasury between 500 coins and 1000 coins; and the web game limits the number of rounds to between 3 and 5.
- **Sampled surveys.** Survey answers changed from exact regional shares to estimates from a 10% sample, so information is noisy and repeating a question can be worthwhile.

3.8 Summary

An electorate is four regions of voters linked by a stochastic block model. Each region has a hidden archetype (what it wants) and a hidden personality (how it can be moved), and 5% of voters are zealots. The player commits a manifesto, then spends money over several rounds on workers, who are stubborn agents that start a one-hop-per-round Independent Cascade, and on regional media. Voters drift towards their friends by DeGroot averaging and vote with a probability that depends on their agreement with the manifesto. The player sees region sizes, free statistics, polls and paid noisy surveys, but never the hidden world. The next chapter describes the computer players that play this game.

Chapter 4

Players and Decision Making

4.1 Overview

A human can play the game through the web interface, but to study it we need players that can play thousands of games without supervision. This chapter describes the computer players we built. All but one follow the same rule as a human: they see only the player's observation, never the hidden state. We first describe the full-information Oracle, which breaks that rule on purpose and serves as a reference. We then describe the Bayesian planner, our strongest observation-only player, and the methods it uses to ask questions, choose a manifesto and plan its spending. We end with the simpler scripted players and a plan search used to estimate how well one can do.

4.2 The player interface

Every player implements one method, $\mathrm{act}(\mathrm{observation})$ -> action. The observation is the same allow-listed JSON object that the web interface receives, and the action is one of four kinds: ask a survey question, commit a manifesto, allocate money for a round, or skip a round. The game engine validates every action; a rejected action changes nothing and costs nothing. Because computer and human players use the same interface, any computer player could, in principle, play through the web API.

4.3 The full-information reference: Oracle

The Oracle is told the true preferences and susceptibilities of every voter. It is not meant to be a fair opponent but a labelled baseline, and its results are never mixed with those of observation-only players.

Manifesto. An important lesson from calibration was that the electorate on election day is not the electorate at the start. Even without any campaign, five rounds of social averaging pull each region towards its mean. The Oracle therefore simulates five rounds of social-only dynamics on the true electorate, which we call the *anticipated* electorate, and optimises the manifesto against that (Section 4.4.3).

Spending. Each round, the Oracle estimates the marginal gain in expected vote share per coin for each possible purchase, using the true state. For media, this is the expected change in P_i over voters who would be reached. For a worker in region r, it estimates the reach of a cascade with per-attempt chance $a_r = \mathbb{E}_r[\gamma P]$ and mean degree $\bar{k}_r$ over the L remaining hops,

$$\mathrm{reach}_r \approx \min\Big(\bar{k}_r a_r \sum_{h=0}^{L-1} (\bar{k}_r a_r)^h,\ \tfrac{1}{2} n_r\Big), \tag{4.1}$$

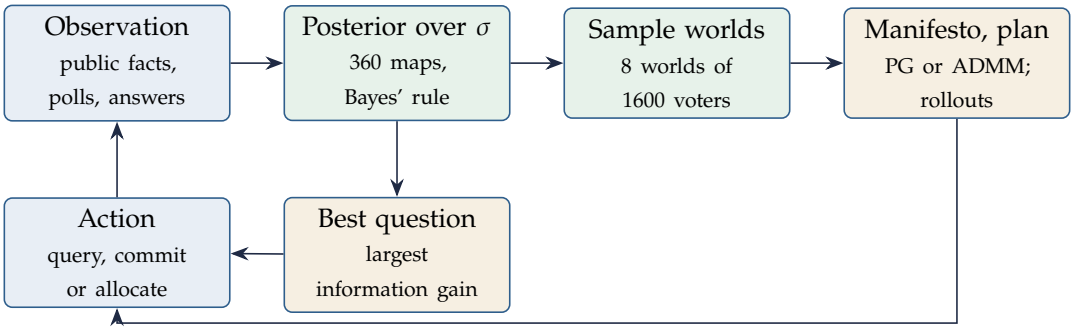


Figure 4.1: One decision of the Bayesian planner. While a question is worth its price it asks one; otherwise it commits a manifesto (in the survey phase) or chooses a round allocation (in a campaign round).

times the expected gain per converted voter. It then buys greedily by value per coin. This is a sensible heuristic, but it is a heuristic: nothing guarantees that it is optimal, and we test that directly in the experiments.

4.4 The Bayesian planner

The Bayesian planner ("player II" in the experiments) is our main observation-only player. Figure 4.1 shows its loop. It keeps a belief about the hidden world, asks the most informative survey questions, picks a manifesto that does well across plausible worlds, and plans each round by simulating the future in worlds drawn from its belief.

4.4.1 Belief over the hidden world

Survey answers and the free top-three statistic depend only on voters' preferences, and initial preferences depend only on the archetype map σ, not on the personalities τ. The planner therefore keeps an exact posterior over all 360 maps σ, starting from a uniform prior, and leaves its belief over τ uniform. For each archetype, the distribution of answers to every possible question is precomputed once by Monte Carlo. When an answer $\hat{a}$ arrives, every map σ is reweighted by a Gaussian likelihood

$$\Pr(\sigma \mid \hat{a}) \;\propto\; \Pr(\sigma)\, \exp\!\Big(-\frac{(\hat{a} - a_\sigma)^2}{2 s_\sigma^2}\Big), \tag{4.2}$$

where a_σ is the answer expected under σ and s_σ combines the sampling error of a 10% sample with the spread of answers across voters.

4.4.2 Which question to ask

A question is a triple (s_1, s_2, r): two sectors and a region. For each of the $10 \times 4 = 40$ possible questions, the planner computes the expected information gain, the mutual information between the answer Y and the archetype map,

$$I(\sigma; Y) = H(\sigma) - \mathbb{E}_Y\big[H(\sigma \mid Y)\big], \tag{4.3}$$

and asks the question with the largest gain. It stops asking when its uncertainty is small (posterior entropy below 0.05 nats), when the best question gains too little per coin, or after ten questions or 50 coins of surveys. During the campaign it may ask one more question per round if the gain is large. Repeating a question is allowed, because each answer is a fresh sample.

4.4.3 Choosing the manifesto

The manifesto should maximise the expected vote share on election day,

$$\max_{b \in \Delta} \; f(b) = \frac{1}{N} \sum_{i} \sigma_{\mathrm{s}}\Big(\beta_v \big(1 - \tfrac{1}{2}\|v_i - b\|_1 - z\big)\Big), \tag{4.4}$$

where σ_{s} is the logistic function and v_i are the voters of the anticipated electorate. The planner does not know the v_i, so it builds eight synthetic worlds from its posterior: each takes a map σ sampled from the belief and draws everything else from the public prior, at a reduced size of 1600 voters that keeps region proportions and degree structure. It then optimises f averaged over these worlds.

The objective is not smooth, because of the absolute values, and not concave. We replace $|x|$ by the pseudo-Huber function $\sqrt{x^2 + \delta^2} - \delta$ with a small δ, and run projected gradient ascent from several starts (uniform, the population mean and the five vertices pulled towards the centre):

$$b^{(k+1)} = P_{\Delta}\big(b^{(k)} + \eta \, \nabla \tilde{f}(b^{(k)})\big), \tag{4.5}$$

where P_{Δ} is the exact Euclidean projection onto the simplex [12]. The best start is chosen on the exact, unsmoothed objective.

The ADMM variant. We also solve the same problem in a distributed way, as if each region negotiated its own preferred manifesto. Writing the objective as a sum over regions, $-f(b) = \sum_r f_r(b)$ with f_r region r's weighted share, consensus ADMM [5] gives each region its own copy x_r and a shared variable z:

$$x_r \leftarrow \arg\min_{x} \; f_r(x) + \tfrac{\rho}{2} \, \|x - z + u_r\|^2, \tag{4.6}$$

$$z \leftarrow P_{\Delta}\Big(\tfrac{1}{4} \textstyle\sum_r (x_r + u_r)\Big), \tag{4.7}$$

$$u_r \leftarrow u_r + x_r - z. \tag{4.8}$$

The first step is each region's local problem, solved inexactly with a few gradient steps; the second is the projection that enforces agreement; the third accumulates each region's disagreement. Because f_r is not convex, ADMM converges to a stationary point and needs a fairly large penalty ($\rho = 5$) and a smoother surrogate ($\delta = 0.02$) to avoid oscillation. A third family of optimisers, mirror descent and Frank–Wolfe, is used only in the experiments that compare optimisers.

4.4.4 Planning each round

Each round the planner builds a short list of at most twenty candidate allocations from public information. The list always includes doing nothing, workers only, media only and mixtures, all affordable and valid. Each candidate is scored by a rollout [4]: in each of the eight sampled worlds, first replay the player's own past actions, then play the candidate this round, then follow a fixed "even" continuation for the remaining rounds (each round spends an equal share of the remaining money, half on media in the least-aired regions and half on workers split by region size). The score is the mean expected vote share on election day over the worlds, and the planner plays the best candidate. All candidates are simulated together in one batched call using common random numbers, so differences between candidates are not drowned out by noise.

4.5 Scripted players

To place the Bayesian planner in context we also built simpler players.

- **People Pleaser (player I).** Uses the same surveys and posterior as the Bayesian planner, but then follows a rule of thumb. It picks the smallest group of the largest regions that together hold a majority, and blends the most likely archetypes of those regions, weighted by region size. It spends at an even pace, with media in its coalition regions and the rest on workers.
- **Strong-Arm (player III).** Asks no questions and pushes a fixed agenda, ignoring what voters want. It spends half on media in the largest regions and half on workers, at one and a half times the even pace, so its money runs out early.
- **Fixed patterns.** Simple spending rules played with a given manifesto: *none* (spend nothing), *all workers*, *all media*, and *split* (half and half at an even pace). They are used to check balance: in a well-designed game, no fixed pattern should win almost always.
- **Top-three and uniform.** Two players that ask nothing: one uses the free top-three statistic as its manifesto and spends evenly, and the other commits the uniform budget and never spends.

An interface for a language-model player also exists: it would receive the same observation as JSON and reply with one action. It is not implemented, because it was outside the scope of this version.

4.6 An estimate of good play: plan search

To know how good a player is we need a target. The ORACLE is one, but it is a heuristic, so we also built an open-loop plan search. Given a manifesto and the true world, it searches over whole five-round plans (how many workers and which media in each region, in each round). It uses the cross-entropy method [42] with 256 plans per iteration, the top 10% as the elite set, 30 iterations and smoothing 0.7, followed by hill climbing. The final plan is re-evaluated on 20 fresh random streams so that its score is not inflated by the noise it was selected on. This search is still a heuristic, so its score is a lower bound on the best achievable, not the optimum itself.

4.7 Summary

All players except the ORACLE see only what a human sees. The ORACLE knows everything, optimises the manifesto against the anticipated electorate and buys greedily by estimated value per coin. The Bayesian planner keeps an exact posterior over the 360 archetype maps, asks the question with the largest expected information gain, optimises the manifesto by projected gradient (or consensus ADMM) on worlds sampled from its belief, and chooses each round's spending by batched rollouts. Scripted players and fixed patterns give simpler points of comparison, and a cross-entropy plan search estimates how well one can do. The next chapter describes how we reason about what each player knew.

Chapter 5

Reasoning About Knowledge and Time

5.1 Overview

After a game, a player wants to know more than whether they won. Was the election already lost when they committed the manifesto? Could they have known how to win? This chapter describes the reasoning layer that answers such questions. We first show how we represent what the player knew at each point in the game as a finite sample of possible worlds. We then define the epistemic properties we evaluate on that sample, and the temporal properties we evaluate over the course of a game. Finally, we describe the debrief that presents these results to a human player.

5.2 Possible worlds

A world w consists of a hidden type $\theta = (\sigma, \tau)$ and a micro-state. Two worlds are indistinguishable for the player, $w \sim w'$, if replaying the player's own actions in both produces the same knowledge log. This relation is an equivalence, so the player's knowledge follows the S5 logic of Chapter 2, and the set of worlds the player considers possible at time t is its information set I_t, which always contains the true world.

The micro-state is continuous, so I_t cannot be enumerated. Instead we draw a finite sample $W_t \subseteq I_t$ by rejection:

1. draw σ from the Bayesian posterior over the 360 archetype maps given all statistics and survey answers so far;
2. draw τ and the micro-state from the public prior, and replay the player's actions;
3. keep the world only if its polls agree with the polls the player saw, that is, for every round,

$$|\hat{p}_w - \hat{p}_{\text{obs}}| \;\le\; z_{\text{poll}} \left(\text{SE}(\hat{p}_{\text{obs}}) + \epsilon\right), \qquad z_{\text{poll}} = 2.5,\ \epsilon = 0.01. \tag{5.1}$$

Because worlds are rescaled to a smaller number of voters for speed, every statement in this chapter is a statement about the sampled worlds, and we always report the sample size with it.

5.3 Epistemic properties

At decision point t, let Π_t be a finite set of plans for the remaining rounds: "spend nothing" and one plan for each candidate allocation, repeated for every remaining round. Playing every plan in every sampled world gives an outcome matrix $V[w, \pi]$, the expected vote share on election day. A plan wins in a world when $V[w, \pi] > 0.5$. We evaluate four properties.

$$\textsc{PossibleVictory}: \quad \exists \pi \in \Pi_t \; \exists w \in W_t : \; V[w, \pi] > 0.5, \tag{5.2}$$

$$\textsc{KnowsWinExists}: \quad \forall w \in W_t \; \exists \pi \in \Pi_t : \; V[w, \pi] > 0.5, \tag{5.3}$$

$$\textsc{GuaranteedVictory}: \quad \exists \pi \in \Pi_t \; \forall w \in W_t : \; V[w, \pi] > 0.5, \tag{5.4}$$

$$\textsc{InevitableDefeat}: \quad \forall \pi \in \Pi_t \; \forall w \in W_t : \; V[w, \pi] \le 0.5. \tag{5.5}$$

The difference between the second and third lines is the difference between *knowing that* a win exists and *knowing how* to win. In the second, the winning plan may differ from world to world, so the player cannot act on it without learning more. In the third, a single plan wins everywhere the player considers possible. Alongside these yes-or-no verdicts we report a graded confidence: the fraction of sampled worlds in which the best plan wins.

To keep knowledge and truth separate, every epistemic field is computed only from the reconstructed observation. A single ground-truth field, whether some plan in Π_t would have won in the true world, is computed separately by replaying the recorded actions through the real engine. It is shown to the player afterwards and is never used by a player during the game.

5.4 Temporal properties

A game produces a short trace, one state per round. Each state carries Boolean atoms and one real signal:

- Lead: the round's poll estimate is above one half;
- BudgetLeft: the treasury is not empty;
- Viable and Guaranteed: PossibleVictory and GuaranteedVictory at that round;
- Victory: the official result (true only in the final state of a won game);
- the margin, the poll estimate minus one half (the official share in the final state).

On these traces we evaluate bounded LTL formulas with the operators next (X), eventually within k steps ($\Diamond_{\leq k}$), always ($\Box$) and until (U). For example, $\neg$Lead $\wedge \Diamond$ Victory says that a player who was behind at the start went on to win. We also compute STL robustness with the usual min-max semantics, where the robustness of $\Diamond_{\leq T}(\text{margin} > 0)$ is the largest margin reached. Over the tree of sampled worlds and plans we evaluate CTL-style properties: $\exists\Diamond\varphi$ (some world and plan eventually reach φ), $\forall\Diamond\varphi$ (all of them do), and the strategic form "some plan reaches φ in every world", which is the ATL statement $\langle\langle p \rangle\rangle \Diamond\varphi$ restricted to Π_t [2].

A useful derived notion is the *point of no return*: the first decision point after which the player's position is no longer viable under any plan in Π_t. It answers the question "when was this game lost?".

5.5 The epistemic debrief

When a human game ends, the web interface shows an *epistemic timeline*. For each round it shows what the player knew at the moment they chose their allocation (the round's poll plus any questions asked that round), whether a win was possible, known to exist or guaranteed over the sampled worlds, the best plan in Π_t, and whether a win was in fact still possible in the true world. A one-line headline summarises the story, for example that the game was already lost at the manifesto commit. The same computation is available to the experiments with extra diagnostics: timing, the number of worlds drawn and accepted, and fallbacks when too few worlds pass the consistency test.

5.6 Summary

The player's knowledge is modelled as an S5 information set, approximated by a finite sample of worlds that agree with everything the player observed. Over this sample and a finite plan set we evaluate whether a win was possible, known to exist, guaranteed, or impossible, and we

keep these knowledge statements strictly separate from the ground truth. Over the course of a game we evaluate LTL, STL and CTL-style properties such as coming from behind and the point of no return. These results drive the post-game debrief. The next chapter describes the software that implements the game, the players and this reasoning layer.

Chapter 6

System Design and Implementation

6.1 Overview

This chapter describes the software. We start with the overall architecture, then explain the three properties we cared about most: that every game can be replayed exactly, that the simulator runs fast enough on a GPU to support large experiments, and that the player never sees hidden information. We then describe the web game, the testing strategy and the experiment harness.

6.2 Architecture

Figure 6.1 shows the layers. At the bottom, a *simulation core* implements electorate generation, dynamics, the campaign mechanisms, voting and the information model as PyTorch [39] tensor operations. Above it, a *game engine* enforces the rules: phases, validation, money, and the order of events in a round. Players, the reasoning layer and the experiment harness all talk to the engine. The web interface reaches it over an HTTP API.

The code is organised as a Python package with one module per concern: `electorate`, `dynamics`, `voting`, `information`, `game`, `strategies`, `logic`, `analysis` (spectral and robustness tools) and `api`. The package has about 5,900 lines of Python, the web client about 5,600 lines of TypeScript, and the experiment scripts about 13,000 lines.

6.3 The game engine

The engine is a set of pure functions over immutable game states. Applying an action returns a new state; a rejected action returns the same state object with an error code (for example `INSUFFICIENT_FUNDS` or `INVALID_PHASE`), so it cannot leave the game half-changed. Money is kept as exact decimals with two places, so prices and treasuries never suffer rounding drift. The engine follows the round order of Fig. 3.1: hires, one cascade hop, media, the social step with workers pinned, then new polls. The election runs automatically after the last round.

6.4 Reproducibility

Every random draw comes from a named stream derived from a root seed: `regions`, `graph`, `preferences`, `attributes`, `zealots`, `workers`, `cascade`, `media`, `poll`, `election` and others. Streams are derived with a cryptographic hash, and per-round draws use sub-seeds keyed by stream, round and player. Two consequences follow.

- The same N and seed produce a bit-identical electorate on any machine with the same PyTorch version, because the electorate is generated from CPU streams and then moved to the device. A player can share a seed and someone else can play the same world.
- A game can be replayed exactly from its seed and its list of actions. The reasoning layer relies on this to compute ground truth at earlier decision points.

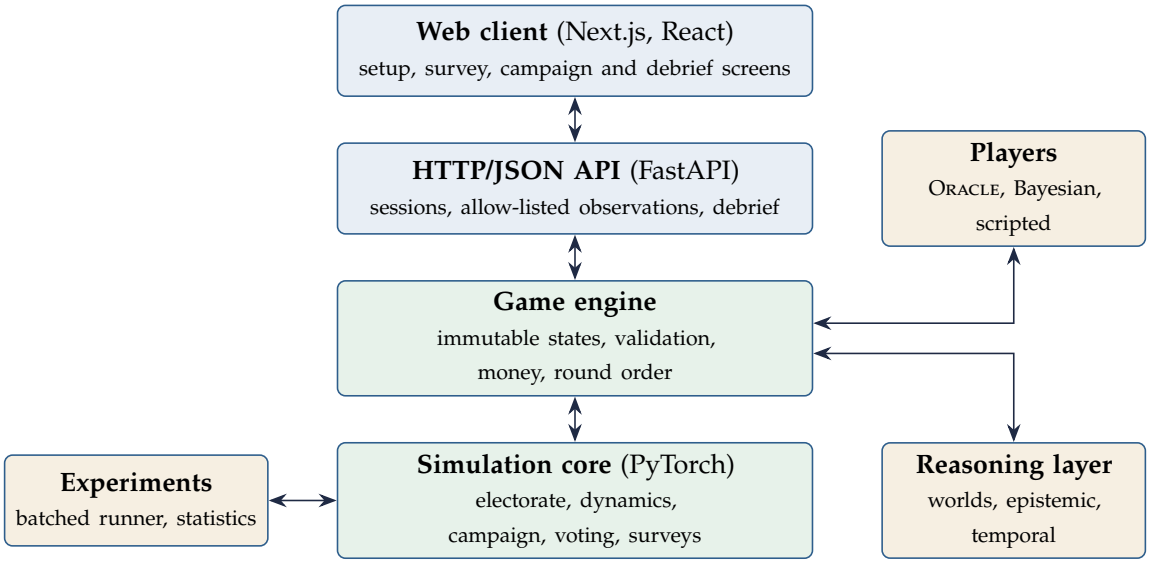


Figure 6.1: Software architecture. Players and the reasoning layer see the game only through observations; the experiment harness drives the simulation core directly for speed.

Every experiment result also records its configuration, seeds, git revision, device, and library versions.

6.5 GPU-first simulation

All operations on voters are tensor operations; there are no Python loops over voters. The graph is stored as an edge list plus a compressed sparse row view, so memory grows with the number of edges and an $N \times N$ matrix is never formed. Every function in the simulation core accepts an optional leading batch dimension, so the same code simulates one world or B worlds at once.

Two uses of batching matter most.

- **Inside players.** The Bayesian planner joins its eight sampled worlds into one disjoint graph and simulates all candidate plans in one call. The reasoning layer does the same for its outcome matrix.
- **In experiments.** A batched true-world runner plays B complete games per call, each with its own world, manifesto, policy and random streams.

The single code path gives a strong correctness check. On the CPU, the batched runner is bit-identical to the reference single-world simulator. On the GPU, it produces identical discrete outcomes (who is hired, who converts, who wins), and its floating-point values agree to within 6×10^{-8}. Every experiment ran a three-seed CPU-to-GPU parity check before its main run. Spectral quantities use SciPy's Lanczos solver on the CPU for moderate N and power iteration on the GPU for large N.

6.6 Keeping hidden information hidden

A game about knowledge is only meaningful if the player cannot peek. Observations, API responses, logs and error messages are built by allow-listed serialisers that copy only public fields. A recursive leak test walks every response and fails if it finds any voter-level preference, susceptibility, zealot flag, edge, archetype or personality map, block probability or seed. Computer

players receive a deep copy of their own observation, so they cannot reach into the engine either. The post-game debrief is the only place where hidden truth is shown, and only after the game is over.

6.7 The web game

Humans play through a browser. The engine is served by a FastAPI [41] application with routes to create a game, fetch the player's observation, ask a question, commit, allocate, skip a round, and fetch the debrief and epistemic timeline. Each game has its own lock, and idle games are evicted when the server is full. The client is a Next.js [44] application with four screens.

1. **Setup:** choose N, the number of rounds, the treasury and an optional seed.
2. **Survey:** read the free statistics, ask questions, and design the manifesto with sliders and a doughnut chart.
3. **Campaign:** a region map, a sample of the friendship network, a poll chart, the round's intelligence, an allocation panel and an optional round timer.
4. **Results:** the outcome, a round-by-round look back, the hidden truth, the ORACLE benchmark and the epistemic timeline.

The network view draws a stratified sample of voters and the edges between them, since drawing ten thousand nodes would be unreadable and would leak more structure than the census allows.

6.8 Testing

The repository contains over two hundred automated tests. They cover the statistical properties of the generators (region floors, mean degree, Dirichlet means), invariants of the dynamics (preferences stay on the simplex, workers and zealots never move), the cascade rules (one hop per round, the three-attempt cap), exact money accounting, the phase machine and its error codes, the privacy leak test, the API contract, the duality and other identities of the epistemic operators, and CPU-to-GPU agreement. Tests that need a GPU skip automatically without one. The web client has unit tests and browser end-to-end tests.

6.9 Experiment harness

The experiments share a small library. Every arm of an experiment runs on the same seeds (a paired design), and different experiment groups use disjoint blocks of seeds. Results are reported with 95% Wilson intervals for rates [46] and t-intervals for means; paired comparisons use the Wilcoxon signed-rank test [45] for differences in vote share and the exact McNemar test [32] for wins, with the Holm–Bonferroni correction [24] within each experiment. Global sensitivity uses the Morris screening design [34].

Before each experiment ran, we wrote down what we expected it to show. Each experiment writes raw per-game rows and a summary file, and every figure is redrawn only from those files. A checking script then reads the written results and confirms that every number in the text appears, after rounding, in the data file it cites.

6.10 Summary

The software is a layered Python package on top of PyTorch, with a game engine of pure functions over immutable states, named random streams that make every game replayable, and a single tensor code path that runs one world on a CPU or many on a GPU with verified parity. Allow-listed serialisers and leak tests keep hidden information hidden. A FastAPI server and a Next.js client make the game playable in a browser, over two hundred tests guard the rules, and a shared harness gives every experiment paired seeds, standard statistics and machine-checked reporting. The next chapter reports what the experiments found.

Chapter 7

Experiments and Results

7.1 Overview

[**TODO:** To be written by the group.]

7.2 Summary

[**TODO:** To be written by the group.]

Chapter 8

Limitations and Future Work

8.1 Overview

No model of an election is complete, and ours makes many simplifying choices on purpose. This chapter lists the limitations that a reader should keep in mind when interpreting the results, and the directions we would take the work next.

8.2 Limitations of the model

- **A synthetic electorate.** Regions, archetypes, personalities and the network are drawn from distributions we chose. The parameters were tuned so that the game is playable and interesting, not fitted to any real electorate. Our conclusions are about this model, not about real elections.
- **One party.** There is no opponent. Voters either support the party or do not, so the game cannot show how two parties' manifestos would compete or converge.
- **Fixed manifesto.** The manifesto cannot be revised after it is committed, and there is no model of turnout beyond abstention.
- **A static network.** Friendships never change during a game, and edges are unweighted.

8.3 Limitations of the method

- **References are heuristics.** The Oracle and the plan search are good players, not provably optimal ones. Any comparison against them measures distance to a strong heuristic, not to the true optimum.
- **Knowledge is sampled.** Epistemic and temporal verdicts are evaluated over finite samples of worlds, plans and futures, at a reduced number of voters. They are statements about the sample.
- **No human data.** All conclusions about play come from computer players. A planned human playtest was dropped for lack of time.
- **Shared hardware.** Experiments ran on shared GPUs, which affects measured timings but not results. CPU and GPU use different random streams, so their results agree statistically, not draw for draw, except where we checked parity.

8.4 Future work

- **A human playtest.** Ask people to play the web game and compare their choices and outcomes with the computer players, and test whether the epistemic debrief helps them play better.
- **A second party.** Add an opponent and a multi-party voting rule, and study whether competing manifestos converge, as spatial voting theory predicts for two parties.

- **A language-model player.** The interface exists; implementing it would let us test how a general-purpose model reasons about a game with hidden information.
- **Exact or certified reasoning.** Replace sampled verdicts with bounds that hold for the whole information set, at least for small electorates.
- **Richer dynamics.** Bounded confidence, changing networks and weighted friendships would test how much our conclusions depend on the simple DeGroot rule.

8.5 Summary

The model is synthetic and has a single party, a fixed manifesto and a static network. Its reference players are heuristics, its knowledge verdicts are sampled, and no human played the final version. The most valuable next steps are a human playtest and a second party.

Chapter 9

Conclusion

9.1 Overview

This chapter draws the report together: what we set out to do, what we built, and what we take away from building it.

9.2 What we did

We set out to study an election campaign as a multi-agent system in which a party pushes on an electorate that is already moving, and does so without seeing it. To make that concrete we built *Dumbocracy*. Its electorate combines DeGroot consensus, zealots and a stochastic block network. Its party workers are stubborn agents that start an Independent Cascade. Its manifesto is a point on the simplex chosen by projected gradient or ADMM. Its information model makes the player's knowledge an S5 information set that can be sampled and reasoned about.

On top of the model we built players that see only what a human sees, including a Bayesian planner that asks questions by information gain and plans by rollouts; a reasoning layer that tells a player, after the game, what they knew and when the game was decided; a web interface; and a GPU-batched simulator that is exact on the CPU and verified on the GPU. Together these turned the game into an instrument for testing course theory with controlled, paired, pre-registered experiments, whose results are reported in Chapter 7.

9.3 What we learned from building it

Some lessons came from building the system rather than from the experiments.

- **The anticipated electorate matters.** Optimising a manifesto against the voters as they are at the start is a mistake, because social averaging moves them before election day. Every good player optimises against where the electorate is going.
- **Mechanics and policy are coupled.** Because a voter's conversion chance depends on how much they already like the manifesto, the choice of manifesto changes how far workers' influence spreads. The "policy" and "network" halves of the game cannot be designed separately.
- **Knowledge needs care to compute honestly.** Keeping what the player knew separate from what was true required explicit reconstruction of earlier observations, a strict privacy boundary and exact replay.
- **One code path pays off.** Writing the simulator once, with a batch dimension, gave us both a reference implementation and a fast one, and made parity a test rather than a hope.

9.4 Summary

We built a game that brings consensus, contagion, optimisation, planning and logic together around one model, and used it to study campaigning on a social network under uncertainty.

The code is public at https://github.com/saikaushhikp/CLOUT, and every result can be reproduced from it.

Bibliography

[1] Daron Acemoglu and Asuman Ozdaglar. Opinion dynamics and learning in social networks. *Dynamic Games and Applications*, 1(1):3–49, 2011.

[2] Rajeev Alur, Thomas A. Henzinger, and Orna Kupferman. Alternating-time temporal logic. *Journal of the ACM*, 49(5):672–713, 2002.

[3] Amir Beck and Marc Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003.

[4] Dimitri P. Bertsekas, John N. Tsitsiklis, and Cynara Wu. Rollout algorithms for combinatorial optimization. *Journal of Heuristics*, 3(3):245–262, 1997.

[5] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[6] Deepayan Chakrabarti, Yang Wang, Chenxi Wang, Jure Leskovec, and Christos Faloutsos. Epidemic thresholds in real networks. *ACM Transactions on Information and System Security*, 10(4):1–26, 2008.

[7] Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304, 1995.

[8] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986.

[9] Morris H. DeGroot. Reaching a consensus. *Journal of the American Statistical Association*, 69 (345):118–121, 1974.

[10] Alexandre Donzé and Oded Maler. Robust satisfaction of temporal logic over real-valued signals. In *Formal Modeling and Analysis of Timed Systems*, volume 6246 of *Lecture Notes in Computer Science*, pages 92–106. Springer, 2010.

[11] Anthony Downs. *An Economic Theory of Democracy*. Harper & Row, New York, 1957.

[12] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning*, pages 272–279, 2008.

[13] David Easley and Jon Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.

[14] James M. Enelow and Melvin J. Hinich. *The Spatial Theory of Voting: An Introduction*. Cambridge University Press, 1984.

[15] Paul Erdős and Alfréd Rényi. On random graphs I. *Publicationes Mathematicae Debrecen*, 6: 290–297, 1959.

[16] Ronald Fagin, Joseph Y. Halpern, Yoram Moses, and Moshe Y. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.

[17] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23 (2):298–305, 1973.

[18] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1–2):95–110, 1956.

[19] Noah E. Friedkin and Eugene C. Johnsen. Social influence and opinions. *Journal of Mathematical Sociology*, 15(3–4):193–206, 1990.

[20] Jacob Goldenberg, Barak Libai, and Eitan Muller. Talk of the network: A complex systems look at the underlying process of word-of-mouth. *Marketing Letters*, 12(3):211–223, 2001.

[21] Benjamin Golub and Matthew O. Jackson. Naïve learning in social networks and the wisdom of crowds. *American Economic Journal: Microeconomics*, 2(1):112–149, 2010.

[22] Rainer Hegselmann and Ulrich Krause. Opinion dynamics and bounded confidence: Models, analysis and simulation. *Journal of Artificial Societies and Social Simulation*, 5(3), 2002.

[23] Paul W. Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social Networks*, 5(2):109–137, 1983.

[24] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[25] Martin Jaggi. Revisiting Frank–Wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning*, pages 427–435, 2013.

[26] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[27] David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 137–146, 2003.

[28] Heath J. LeBlanc, Haotian Zhang, Xenofon Koutsoukos, and Shreyas Sundaram. Resilient asymptotic consensus in robust networks. *IEEE Journal on Selected Areas in Communications*, 31(4):766–781, 2013.

[29] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 420–429, 2007.

[30] Dennis V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

[31] Oded Maler and Dejan Nickovic. Monitoring temporal properties of continuous signals. In *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, volume 3253 of *Lecture Notes in Computer Science*, pages 152–166. Springer, 2004.

[32] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[33] Mauro Mobilia. Does a single zealot affect an infinite group of voters? *Physical Review Letters*, 91(2):028701, 2003.

[34] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991.

[35] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1): 265–294, 1978.

[36] Reza Olfati-Saber, J. Alex Fax, and Richard M. Murray. Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*, 95(1):215–233, 2007.

[37] Neal Parikh and Stephen Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014.

[38] Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3):925–979, 2015.

[39] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[40] Amir Pnueli. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, pages 46–57, 1977.

[41] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com, 2026. Accessed November 2026.

[42] Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

[43] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

[44] Vercel. Next.js: The React framework. https://nextjs.org, 2026. Accessed November 2026.

[45] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6): 80–83, 1945.

[46] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[47] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

Chapter A

Notation and Default Parameters

Notation

Table A.1: Notation used in the report.

Symbol	Meaning
N, T	number of voters; number of campaign rounds
r, n_r	region index (North, South, East, West); number of voters in region r
Δ	the probability simplex of budget splits over five sectors
$v_i \in \Delta$	voter i's preferred budget split (hidden)
$b \in \Delta$	the player's manifesto
σ, τ	hidden maps from regions to archetypes and to personalities
$\theta = (\sigma, \tau)$	hidden type of a world (8640 possibilities)
$\alpha_i, \beta_i, \gamma_i$	susceptibility to friends, media and canvassing
S_i	agreement with the manifesto, $1 - \frac{1}{2}\|v_i - b\|_1$
P_i	probability that voter i votes for the party
E	expected vote share, the mean of P_i
A, D, L	adjacency, degree and Laplacian matrices of the friendship graph
W_t, Π_t	sampled consistent worlds and plan set at decision point t
K, M	knowledge and possibility operators

Default parameters

Table A.2: Default parameters of the v3 game.

Parameter	Symbol	Default
Number of voters	N	10^4 (allowed 10^3 to 10^6)
Campaign rounds	T	5 (web game 3 to 5)
Treasury	–	500 coins (allowed 500 to 1000)
Minimum region share	–	15%
Mean degree	$\bar{k}$	30
Within-region link weight	p_r	$U[0.5, 0.9]$
Between-region link weight	p_{rs}	$U[0.01, 0.2]$
Dirichlet concentration	κ	60
Zealot fraction	ζ	0.05
Social step	ω	0.15
Worker price	–	20 coins, paid once
Worker minimum degree; attempt cap	–	3; 3 per voter per game
Media base price	m_0	50 coins per quarter-sized region
Media reach; strength; decay	p_m, η_0, λ	0.6; 0.25; 0.5
Vote slope; threshold	β_v, z	20; 0.825
Survey price; sample fraction	–	10 coins; 10% of the region
Free poll size	–	400 voters

Chapter B

Reproducing the Work

All code is in the public repository https://github.com/saikaushhikp/CLOUT. The commands below assume a Python 3.11 or newer environment and, for the web game, Node.js 22.

Install and test.

```
pip install -e '.[web,analysis,test]'
python -m pytest -q          # GPU tests skip without CUDA
```

Play the game.

```
scripts/run_game.sh          # engine on :61008, web client on :61007
```

Run an experiment and redraw the figures. Each experiment is one script in the experiments folder, for example x6_tournament.py for the strategy tournament. The exact command line of every run is stored in its result file, in the field provenance.command.

```
python experiments/x6_tournament.py --device cuda:0
python experiments/plot_v3.py
python experiments/x_report_check.py
```

The last command checks every number in the results write-up against the data file it cites.